

CSA1717-ARTIFICIAL INTELLIGENCE
PRACTICAL LAB EXPERIMENTS
RAGAVI S – (192565027)

EXPERIMENT 11: MAP COLORING

Aim

To write a Python program to implement the Map Coloring problem using Constraint Satisfaction Problem (CSP).

Objective

- To color a map such that no two adjacent regions have the same color.

Algorithm

1. Start.
2. Define the map and neighboring regions.
3. Assign colors to each region.
4. Check whether adjacent regions have different colors.
5. If the coloring is valid, display the result.
6. Stop.

Source Code

```
def is_valid(node, color, assignment, graph):  
    for neighbor in graph[node]:  
        if neighbor in assignment and assignment[neighbor] == color:  
            return False  
    return True
```

```

def backtrack(assignment, graph, colors, nodes):
    if len(assignment) == len(nodes):
        return assignment

    # Pick the next unassigned node
    unassigned = [n for n in nodes if n not in assignment]
    node = unassigned[0]

    for color in colors:
        if is_valid(node, color, assignment, graph):
            assignment[node] = color
            result = backtrack(assignment, graph, colors, nodes)
            if result:
                return result
            del assignment[node] # Backtrack

    return None

# Problem Setup
graph = {
    "A": ["B", "C"],
    "B": ["A", "C", "D"],
    "C": ["A", "B", "D"],

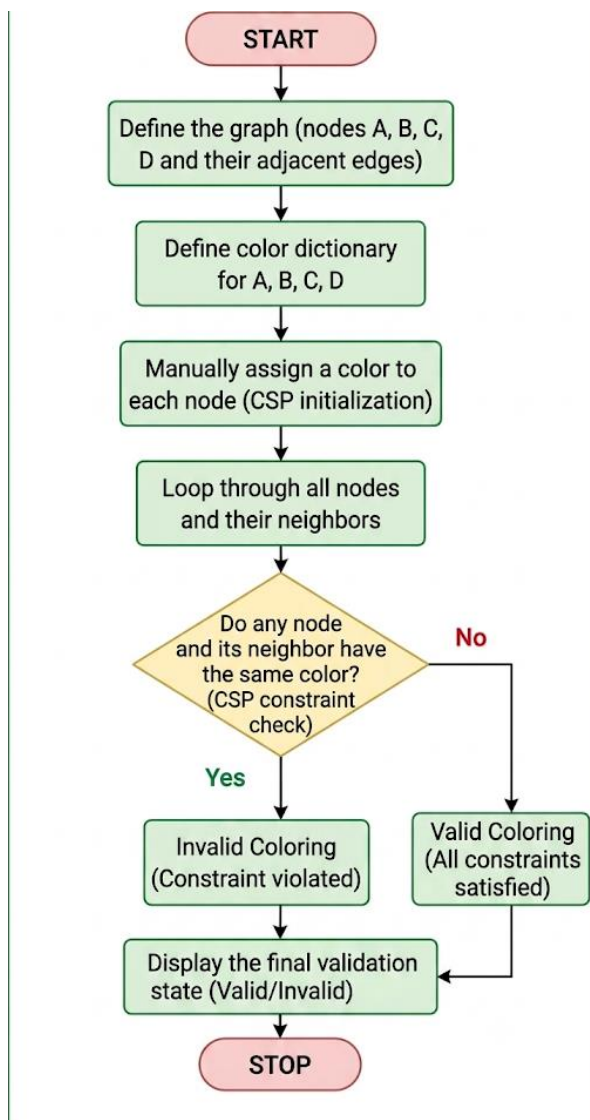
```

```
    "D": ["B", "C"]
}
available_colors = ["Red", "Green", "Blue"]
nodes = list(graph.keys())

solution = backtrack({}, graph, available_colors, nodes)

if solution:
    print("Valid Coloring Found:")
    print(solution)
else:
    print("No valid coloring possible.")
```

Flowchart



Output

```
>>>|===== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/Q.py =====|
|Valid Coloring Found:|
|{'A': 'Red', 'B': 'Green', 'C': 'Blue', 'D': 'Red'}|
>>>|
```

Result

Thus, the Python program for Map Coloring using CSP was executed successfully.